

CatalogGuard Lite

핵심 면접 치트시트

Python·FastAPI 백엔드 프로젝트 · 면접 직전 10 분 복습용

30 초 소개

공급사마다 다른 상품 CSV 를 JSON Profile 기반 공통 ETL 로 표준화하고, 데이터 품질 검수 후 정상 데이터만 PostgreSQL staging 에 적재하는 FastAPI 백엔드 MVP 입니다. 운영 반영은 Preview → 사용자 승인 → hash 재검증 → transaction → Audit 순서로 처리하고, 잘못된 Promotion 은 최신 상태 충돌을 확인한 뒤 Rollback 할 수 있게 구현했습니다.

A. 프로젝트 · 구조

첫 질문에서는 기술 이름보다 “어떤 문제를 어떤 안전장치로 해결했는가”를 먼저 말합니다.

Q1. 어떤 프로젝트인가요?

핵심 패션·이커머스 상품 카탈로그의 품질 오류를 검수하고, 공급사 CSV 를 공통 형식으로 표준화한 뒤 staging → Preview/승인 → 운영 반영 → Rollback/Audit 까지 연결한 Python·FastAPI 백엔드 MVP 입니다.

쉽게 상품 엑셀을 바로 운영에 넣지 않고 서버가 검사·분리·확인·반영·되돌리기까지 관리합니다.

Q2. 왜 FastAPI 인가요?

핵심 데이터 처리 로직이 Python·pandas 중심이어서 같은 언어로 API 와 검수 로직을 연결하기 쉬웠고, Pydantic 기반 요청·응답 계약과 명확한 라우팅 구조가 MVP 에 적합했습니다.

쉽게 FastAPI 가 항상 Django 보다 좋다는 뜻은 아니며, 관리자·ORM·인증이 큰 서비스라면 Django 도 좋은 선택입니다.

Q3. 왜 검수는 FastAPI 한 곳에서만 실행하나요?

핵심 Streamlit 과 서버가 각각 검수하면 기준이 달라지고 같은 CSV 를 두 번 처리할 수 있어, FastAPI 를 단일 검수 기준으로 두고 Streamlit 은 서버 결과만 표시하도록 했습니다.

쉽게 채점 기준을 한 곳에 둔 것입니다.

B. ETL · 데이터 품질

Q4. 공급사마다 CSV 가 다른 문제는?

핵심 공급사별 컬럼명·기본값 차이를 JSON Profile 로 분리하고, 공통 transformer/pipeline 이 Profile 을 읽어 표준 schema 로 변환하도록 했습니다.

쉽게 공급사마다 Python 코드를 복제하지 않고 설정표만 추가합니다.

꼬리질문 두 번째 Profile 추가 때 공통 ETL 코드를 수정하지 않은 점을 말합니다.

Q5. staging 을 왜 따로 두었나요?

핵심 외부 변환 데이터를 바로 운영 상품에 쓰지 않고 검증·조회·Preview 를 거치는 중간 안전 구역이 필요했기 때문입니다.

쉽게 매장 진열 전 검수 참고입니다.

Q6. reject 데이터는 어떻게 처리하나요?

핵심 변환·검증 실패 행은 normal 과 섞지 않고 reject 로 분리하고, 이유를 저장해 조회할 수 있게 했습니다. 개인정보 의심 값은 마스킹해서 노출을 줄였습니다.

쉽게 불량품을 버리지 않고 이유와 함께 별도 상자에 둡니다.

2 DB·동시성·운영 반영

PostgreSQL 을 쓴 이유보다 “DB 가 어떤 무결성을 책임지게 했는가”가 핵심입니다.

A. PostgreSQL · 중복 방지

Q7. PostgreSQL 을 왜 선택했나요?

핵심 검수 이력, ETL batch, staging, 운영 상품, Promotion/Audit 처럼 관계와 제약이 중요한 데이터가 많아 transaction · FK · CHECK · unique/index 를 활용할 수 있는 관계형 DB 가 적합했습니다.

쉽게 잘못 저장되지 않게 DB 도 규칙을 지키게 했습니다.

Q8. 같은 CSV 를 다시 올리면?

핵심 CSV bytes 의 SHA-256 + inspection_version 으로 기존 결과를 사전 조회해 재검수와 신규 저장을 생략합니다. 동시에 들어오는 경쟁 요청은 PostgreSQL partial unique index 가 최종 차단합니다.

쉽게 사전 조회는 성능 최적화, DB unique 는 최종 무결성입니다.

Q9. 사전 조회와 DB unique 가 둘 다 필요한 이유는?

핵심 두 요청이 동시에 “없음”을 볼 수 있기 때문입니다. 애플리케이션 조회만으로 race condition 을 막을 수 없어 DB 제약을 마지막 안전장치로 둡니다.

쉽게 두 사람이 동시에 빈 좌석을 봐도 최종 예약은 좌석 시스템이 막아야 합니다.

Q10. transaction 은 왜 필요한가요?

핵심 ETL 적재 · Promotion · Rollback 처럼 여러 DB 변경 중 일부만 성공하면 데이터가 어긋나므로 하나의 transaction 으로 묶고 실패 시 전부 취소합니다.

쉽게 송금에서 출금만 되고 입금만 안 되는 상태를 막습니다.

B. Promotion

Q11. Promotion 은 무엇인가요?

핵심 할인 행사가 아니라 staging 의 검증된 상품을 운영 catalog 에 반영하는 업무 단계입니다.

쉽게 검수 창고의 상품을 실제 매장에 진열하는 단계입니다.

Q12. 왜 Preview 가 필요한가요?

핵심 실제 DB 변경 전에 insert/update/no-change 와 변경 전 · 후 값을 보여줘 사용자가 영향 범위를 확인하도록 합니다. Preview 는 읽기 단계이고 실제 쓰기 transaction 은 실행 때 시작합니다.

Q13. expected_preview_hash 는 무엇인가요?

핵심 사용자가 확인한 canonical Preview 의 SHA-256 입니다. 실행 직전 서버가 Preview 를 다시 계산해 hash 가 같을 때만 반영하고 다르면 409 preview_stale 로 차단합니다.

쉽게 승인한 견적서와 실행 직전 견적서가 같은지 확인합니다.

꼬리질문 hash 는 암호화가 아니라 승인 상태의 동일성을 확인하는 무결성 장치입니다.

C. Rollback

Q14. 과거 Promotion 을 안전하게 되돌리는 방법은?

핵심 원본 Promotion Audit 의 after_data 와 현재 catalog business field 를 비교합니다. 다르면 이후 정상 변경이 있다고 보고 rollback_conflict 로 차단하며, 같을 때만 Preview/확인/hash 재검증 후 transaction 으로 되돌립니다.

쉽게 옛날 되돌리기가 더 최신 수정값을 덮어쓰지 못하게 합니다.

Q15. INSERT 와 UPDATE 의 Rollback 차이는?

핵심 Promotion 에서 새로 INSERT 한 상품은 당시 after 와 현재가 같으면 삭제하고, UPDATE 한 상품은 Audit 의 before 상태로 복원합니다. 원본 Promotion Audit 은 지우지 않고 별도 Rollback Run/Change Audit 을 남깁니다.

Q16. Rollback 중간에 실패하면?

핵심 여러 상품을 하나의 session.begin() transaction 으로 처리해 하나라도 실패하면 상품 변경과 Rollback Change 를 모두 취소합니다. 같은 Promotion 의 성공 Rollback 은 service 검사와 partial unique index 로 중복 실행도 막습니다.

3 Auth·비동기·테스트·실제 버그

면접에서 가장 강한 이야기는 “테스트 수”가 아니라 실제 결함을 재현하고 구조적으로 고친 경험입니다.

A. Authentication · RBAC · Audit

Q17. Authentication 과 RBAC 는 어떻게 했나요?

핵심 PostgreSQL users 에 bcrypt password_hash · role · is_active 를 저장하고 로그인 시 JWT Access Token 을 발급합니다. FastAPI 는 token 의 sub 로 사용자를 DB 에서 다시 조회한 뒤 require_viewer/require_operator 로 endpoint 권한을 강제합니다.

쉽게 로그인은 누구인지 확인, RBAC 는 어디까지 할 수 있는지 확인입니다.

Q18. Streamlit 에서 버튼만 숨기면 왜 부족하나요?

핵심 UI 는 직접 API 호출로 우회할 수 있어 실제 보안 경계가 아닙니다. viewer 의 버튼을 비활성화해도 FastAPI require_operator 가 서버에서 403 으로 차단합니다.

쉽게 화면이 아니라 서버 출입문에서 권한을 확인합니다.

꼬리질문 토큰 없음/무효/만료/inactive 는 401, 로그인했지만 권한 부족은 403 입니다.

Q19. Actor Audit 은 왜 필요한가요?

핵심 RBAC 가 “누가 실행할 수 있는가”를 통제한다면 Actor Audit 은 “실제로 누가 실행했는가”를 남깁니다. actor 는 request body 가 아니라 인증된 current_user 에서 가져오고 username snapshot 을 기록합니다.

B. Redis · Celery

Q20. Redis 와 Celery 를 왜 썼나요?

핵심 CSV 검수처럼 길어질 수 있는 작업을 HTTP 요청 수명에서 분리하기 위해 사용했습니다. Redis 는 broker/상태 저장, Celery Worker 는 실제 검수, 영속 결과는 PostgreSQL 이 담당합니다.

쉽게 접수 창구와 실제 작업자를 분리합니다.

꼬리질문 작업이 매우 짧다면 동기 처리나 BackgroundTasks 가 더 단순할 수 있습니다.

C. 테스트 · CI

Q21. 테스트가 1,000 개 넘는 게 중요한가요?

핵심 아닙니다. 개수보다 실제 결함을 잡고 재발을 막는지가 중요합니다. 1262 개가 모두 통과하던 상태에서도 실제 Sync Inspection route 의 PostgreSQL Session 충돌이 남아 있었고 route-level regression test 로 보완했습니다.

Q22. 가장 기억에 남는 버그는?

핵심 같은 request Session 에서 사전 SELECT 가 SQLAlchemy autobegin transaction 을 연 뒤 저장 Service 가 session.begin()을 다시 호출해 InvalidRequestError 500 이 발생했습니다. 사전 중복 조회를 별도 precheck Session 으로 분리해 저장 Service 가 유일한 transaction owner 를 유지하도록 고쳤습니다.

쉽게 임시 commit/rollback 으로 숨기지 않고 transaction ownership 을 바로잡았습니다.

Q23. Chromium Browser E2E 까지 만든 이유는?

핵심 API/AppTest 가 통과해도 DOM · accessible label · checkbox · 배치 선택처럼 실제 브라우저에서만 드러나는 문제가 있어 사용자 클릭 흐름과 PostgreSQL 최종 상태까지 검증했습니다.

쉽게 부품 테스트와 실제 완성차 주행 테스트의 차이입니다.

Q24. 현재 CI 는 무엇을 검증하나요?

핵심 main CI 는 test, browser-e2e, kubernetes-smoke, terraform-validate, airflow-smoke 5 개 job 으로 분리되어 있습니다. 일반 pytest 뿐 아니라 PostgreSQL/Redis, Chromium, kind, Terraform validate/mock, Airflow DAG 까지 각각 검증합니다.

쉽게 정확한 숫자보다 검증 범위를 말합니다.

4 인프라 범위·한계·10 초 암기

과장하지 않는 답변이 중요합니다. “구현/검증한 범위”와 “실서비스 운영 경험”을 분리해서 말합니다.

A. Infra 질문 - 범위를 정확히

Q25. Docker build 성공만으로 왜 부족했나요?

핵심 이미지가 빌드돼도 실행에 필요한 package 가 빠질 수 있습니다. 실제로 etl package 누락으로 runtime import 가 실패했고, CI 에서 image import·UID 10001·migration·Uvicorn startup·/health/readiness 를 실행해 막았습니다.

쉽게 상자가 만들어졌다고 제품이 켜지는 것은 아닙니다.

Q26. Kubernetes·Terraform·Airflow 는 어디까지 했나요?

핵심 Kubernetes 는 kind smoke, Terraform 은 fmt/init/validate 와 mock provider test, Airflow 는 manual configured HTTP feed DAG 와 staging/idempotency/lineage 검증까지입니다. production cluster 운영, 실제 Terraform CI apply, 상시 스케줄 운영까지 했다고 말하지 않습니다.

Q27. 현재 가장 큰 한계는?

핵심 Refresh Token·회원가입/reset·OAuth/MFA/SSO·rate limit·세밀한 ACL, streaming/증분 ETL, 대규모 동시 트래픽/SLA, production autoscaling 은 현재 범위 밖입니다. 다음 기능은 목표 공고에서 반복되는 요구와 연결되는 한 가지를 선택하겠습니다.

Q28. unknown size 를 왜 자동 오류로 만들지 않나요?

핵심 사전에 없는 size 라는 이유만으로 잘못된 값이라고 단정하기 어렵기 때문에 미판정으로 유지합니다. TOP/BOTTOM/OUTER/SHOES 는 size 필수, BAG 은 선택이며 빈 값과 FREE 는 서로 다른 값으로 유지합니다. fuzzy matching 은 사용하지 않습니다.

B. 10 초 핵심 문장

키워드	10 초 답변
프로젝트	공급사 CSV 표준화 + 품질 검수 + staging + Preview 승인 Promotion + 안전한 Rollback/Audit 을 연결한 FastAPI 백엔드 MVP.
동일 CSV	SHA-256 + inspection_version 으로 재검수 생략, DB partial unique index 로 경쟁 요청 최종 차단.
ETL	JSON Profile 로 공급사 차이를 설정화하고 Web/CLI 가 같은 ETL Core 를 재사용.
Transaction	ETL/Promotion/Rollback 중 일부 실패 시 데이터가 반만 남지 않도록 전체 원자적 취소.
Promotion	Preview 에서 영향 확인 → 승인 → hash 재검증 → transaction → Audit.
Rollback	현재 state 와 원 Promotion after 를 비교해 최신 변경을 보호하고, 충돌 없을 때만 되돌림.
RBAC	JWT + users 재조회. viewer 는 조회/Preview, operator 는 쓰기. 최종 권한은 FastAPI 가 강제.
테스트	테스트 개수보다 실제 PostgreSQL/Browser/Runtime 결함을 재현하고 회귀 테스트로 막은 경험이 핵심.

C. 모르는 질문을 받았을 때

- “제가 직접 구현한 범위에서는 A 까지 확인했습니다. B 는 현재 프로젝트에서 사용하지 않아 정확한 동작은 확인이 필요합니다.”
- 추측을 사실처럼 채우지 말고, 알고 있는 원리와 확인해야 할 부분을 나눠 말합니다.
- 모르는 기술 질문은 “다만 CatalogGuard Lite 에서는 비슷한 문제를 transaction/constraint/회귀 테스트로 해결했습니다.”처럼 실제 경험으로 돌아옵니다.
- 틀린 것을 깨달으면 바로 “방금 설명한 부분은 정확하지 않았습니다. 정정하면 …입니다.”라고 수정합니다.

마지막 한 문장 기능 수보다 데이터 무결성, transaction 경계, 실제 실행 환경을 검증하고 설명할 수 있는 백엔드 개발자가 되고 싶습니다.